

Lecture Homework Week 07 - Thursday

Template

Student Information

Name: Abhiram Kamisetti

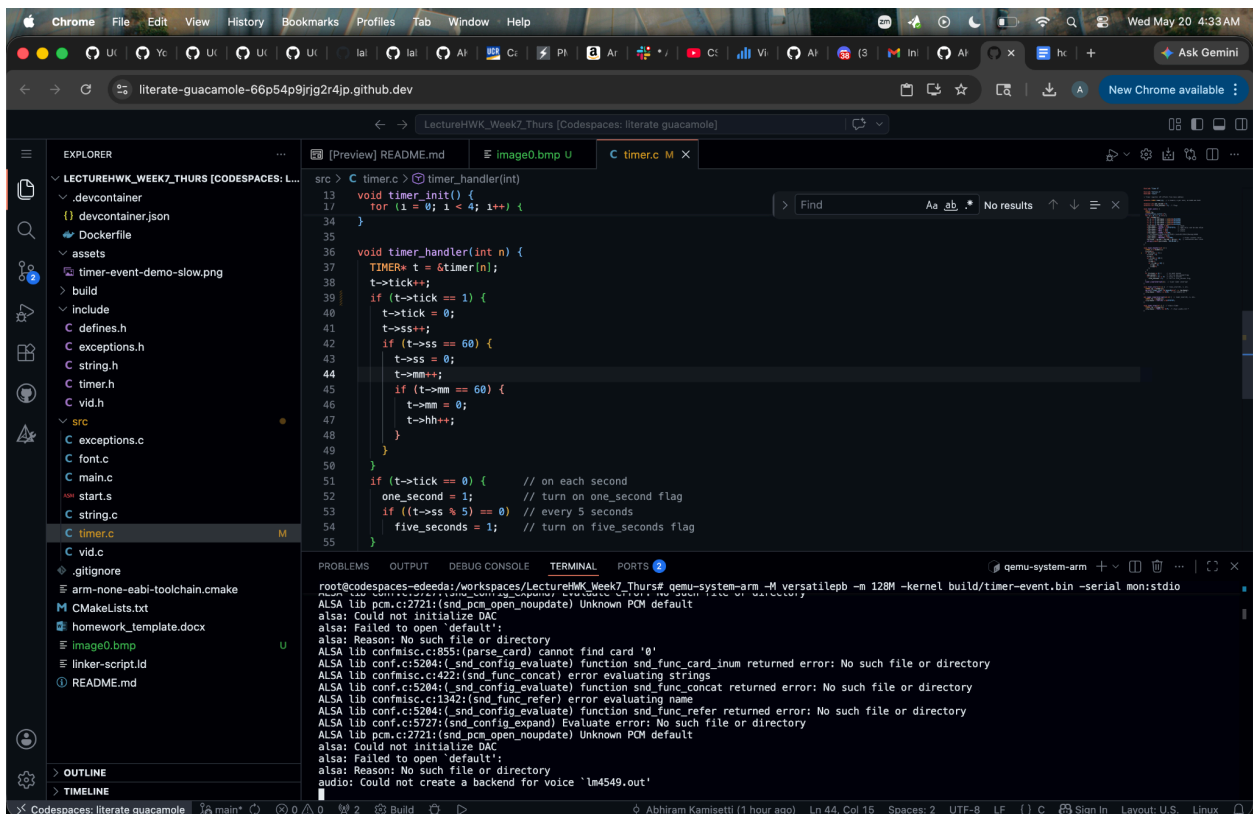
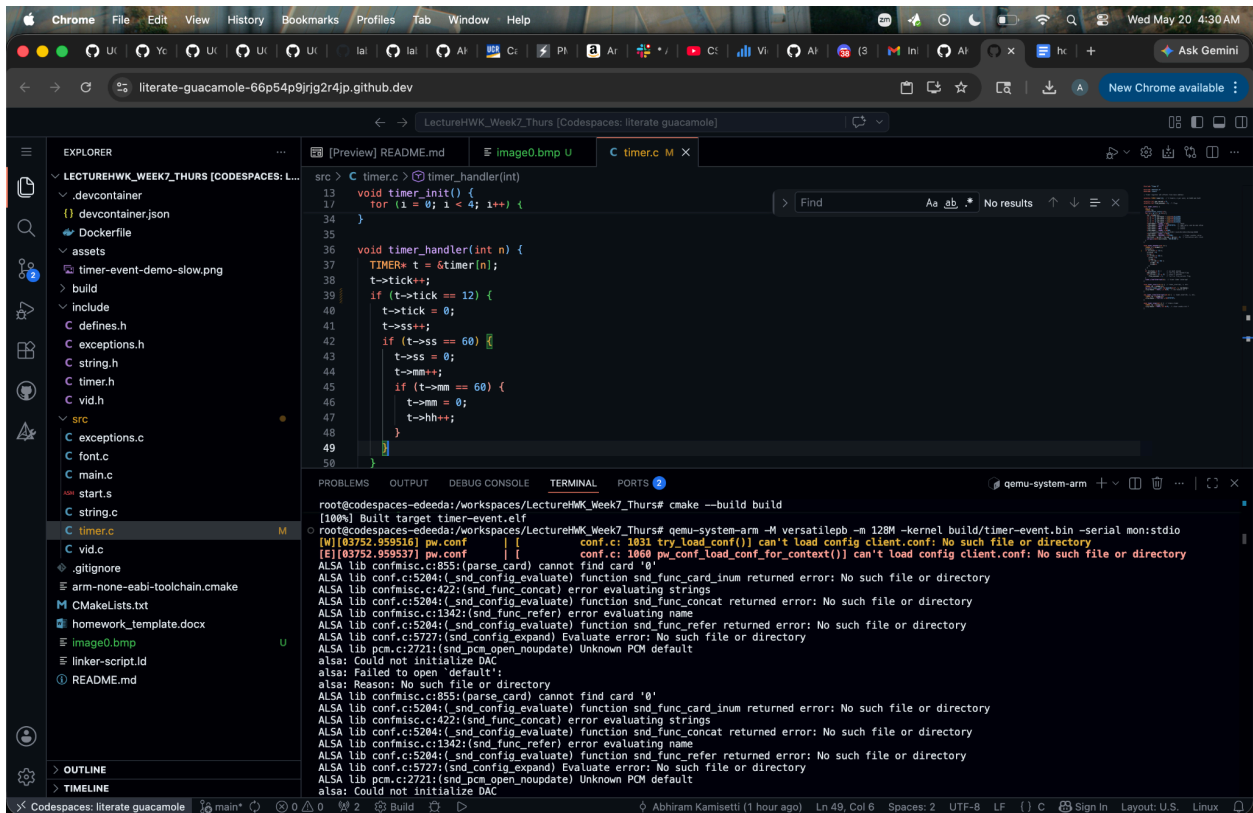
NetID: akami006

Student Responses

Answer the following questions

Question 1

Comment out the `asm` line. Run the program again to see what would happen. What do you observe that happens differently than before :



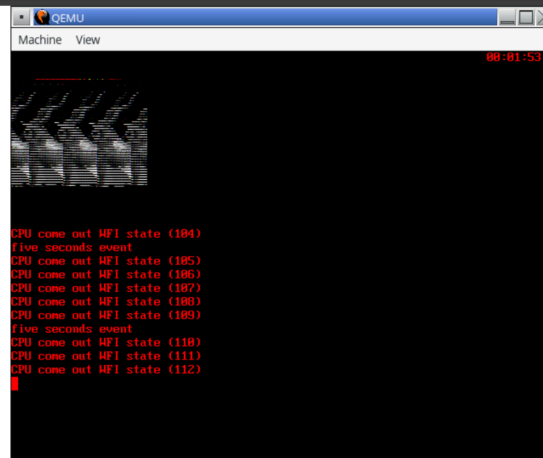
Chrome browser window showing the GitHub repository `literate-guacamole-66p54p9jrg2r4jp.github.dev`. The code editor displays the `timer.c` file, which implements a timer handler. The code includes initialization for a timer, setting up a loop for four iterations, and configuring the timer's base address, value, and status. The timer is initialized to 0x1000, and the wall clock is initialized to 00:00:00.

```
13 void timer_init() {
14     for (i = 0; i < 4; i++) {
15         tp = &timer[i];
16         if (i == 0) tp->base = (u32*)0x101E2000;
17         if (i == 1) tp->base = (u32*)0x101E2020;
18         if (i == 2) tp->base = (u32*)0x101E3000;
19         if (i == 3) tp->base = (u32*)0x101E3020;
20         *(tp->base + TLOAD) = 0x0; // reset
21         *(tp->base + TVALUE) = 0xFFFFFFFF; // read only; can be any value
22         *(tp->base + TRIS) = 0x0; // status
23         *(tp->base + TMIS) = 0x0; // status
24         *(tp->base + TLOAD) = 0x100;
25         // CntlReg=011=|En|Pe|IntE|-|scal=01|32bit|0=wrap|=0x66
26         *(tp->base + TONTL) = 0x66;
27         *(tp->base + TBGLOAD) = 0x10000; // timer counter value
28         tp->tick = tp->hh = tp->mm = tp->ss = 0; // initialize wall clock
29         strcpy((char*)tp->clock, "00:00:00");
30     }
31 }
```

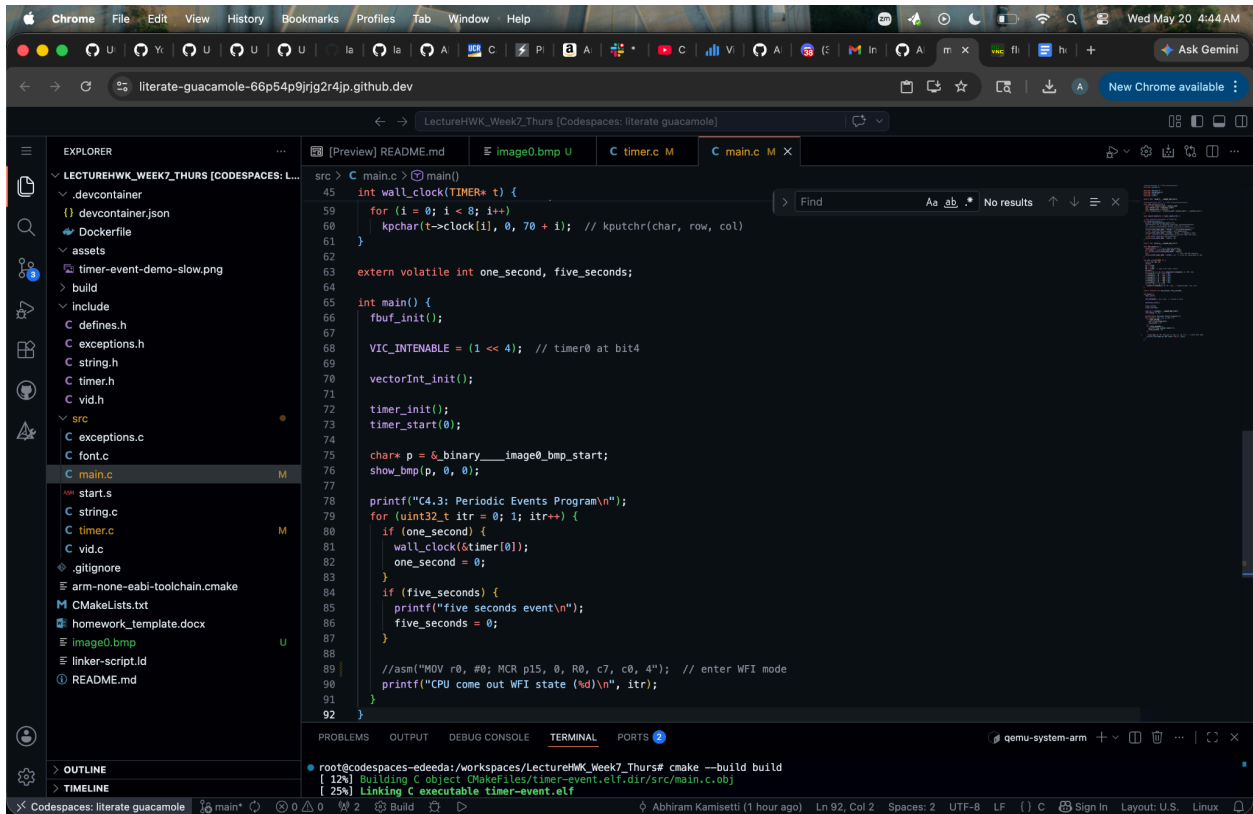
The bottom panel shows the **PORTS** section with two ports:

Port	Forwarded Address	Running Process	Visibility	Origin
5901	https://literate-guacam...	/usr/bin/Xtigervnc -l -localhost=1 -desktop fluxbo...	Private	Codespaces: literate guacamole
6080	https://literate-guacam...	python3 -m websockify --web /usr/local/novnc/no...	Private	Codespaces: literate guacamole

Chrome browser window showing the GitHub repository `literate-guacamole-66p54p9jrg2r4jp-6080.app.github.dev`.

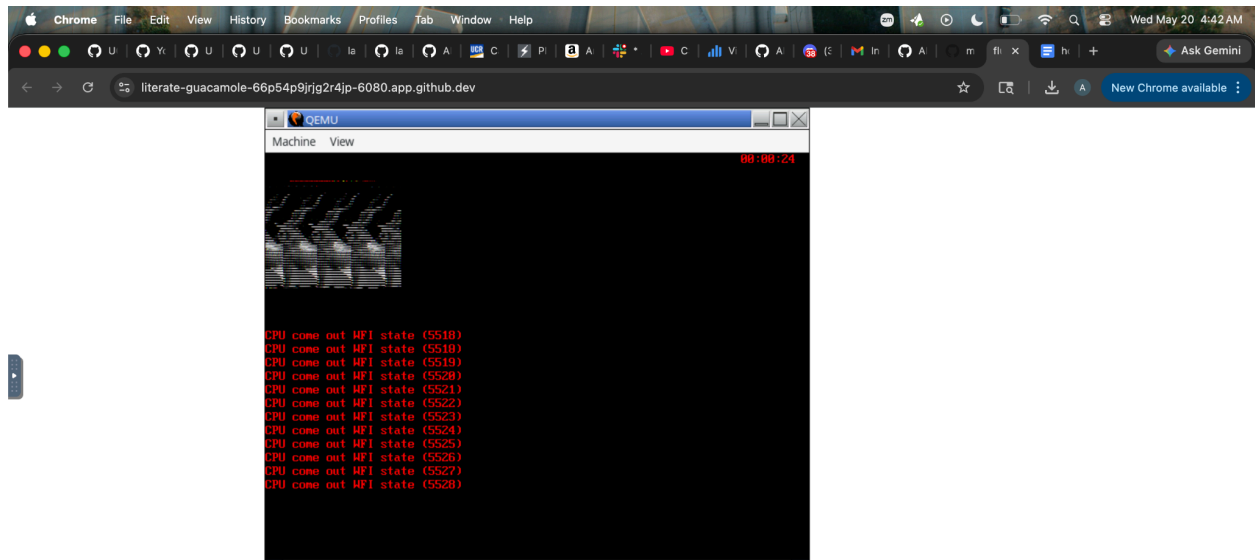


After excluding by commenting:



```
src > C main.c > main()
45 int wall_clock(TIMER* t) {
59     for (i = 0; i < 8; i++)
60         kpchar(t->clock[i], 0, 70 + i); // kpchr(char, row, col)
61     }
62
63     extern volatile int one_second, five_seconds;
64
65     int main() {
66         fbuf_init();
67
68         VIC_INTENABLE = (1 << 4); // timer0 at bit4
69
70         vectorInt_init();
71
72         timer_init();
73         timer_start(0);
74
75         char* p = &_binary_image0_bmp_start;
76         show_bmp(p, 0, 0);
77
78         printf("C4.3: Periodic Events Program\n");
79         for (uint32_t itr = 0; i; itr++) {
80             if (one_second) {
81                 wall_clock(&timer[0]);
82                 one_second = 0;
83             }
84             if (five_seconds) {
85                 printf("five seconds event\n");
86                 five_seconds = 0;
87             }
88
89             //asm("MOV r0, #0; MCR p15, 0, R0, c7, c0, 4"); // enter WFI mode
90             printf("CPU come out WFI state (%d)\n", itr);
91         }
92     }
```

root@codespaces-edeeda:/workspaces/LectureHWK_Week7_Thurs# cmake --build build
[12%] Building C object CMakeFiles/timer-event.elf.dir/src/main.c.obj
[25%] Linking C executable timer-event.elf



When I commented out the asm line, it means that we are not in the WFI low-power state anymore. This means that rather than waiting for an interrupt, it kept looping non stop. Before I commented it seemed as if the CPU waited in WFI mode and resumed only after a timer interrupt woke it up. But after the comment the message CPU came out of WFI state and repeatedly kept printing very quickly while the counter kept increasing.

Question 2

In terms of CPU power consumption, what difference does the asm statement make?

The statement allows the CPU to wait for an interrupt. This means the CPU “sleeps” until it is woken up by a timer.